Lago AI Agent

Natural Language Interface for Billing Operations

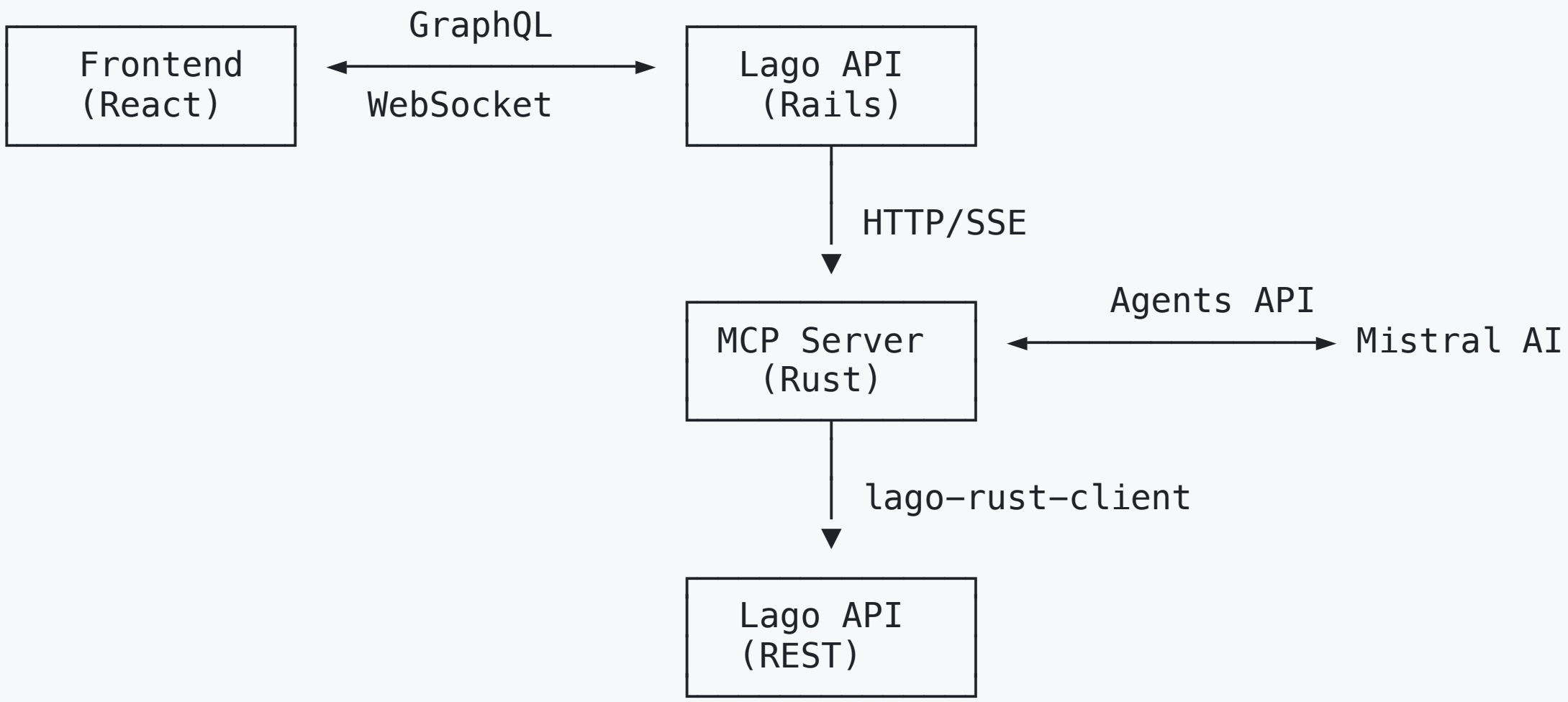
Query invoices, customers, payments, and more using conversational AI

Why an AI Agent?

- Billing data is complex and spread across multiple entities
- Users often need quick answers: *"Show me overdue invoices"*
- Traditional UIs require navigation through multiple screens
- API queries require technical knowledge

Solution: Natural language interface powered by AI

System Overview



Technologies

Component	Technology	Purpose
Frontend	React + Apollo Client	Chat UI & GraphQL subscriptions
Backend	Rails + Action Cable	GraphQL API & WebSocket streaming
MCP Server	Rust + rmcp	Model Context Protocol server
AI Model	Mistral Agents API	LLM with function calling
API Client	lago-rust-client	Type-safe Lago API interactions

How It Works - User Perspective

1. **User types:** *"List the latest activity logs"*
2. **AI responds** with formatted, actionable data
3. **Clickable links** to invoices, customers, etc.
4. **Conversation history** maintained for context

Example Response:

> **Invoice Payment Overdue**
>
> - Activity Type: `invoice.payment_overdue`
> - Customer: `Test Customer DZ`
> - Amount: 240.00 DZD

Technical Flow

```
1. User sends message
       │
       ▼
2. Frontend → GraphQL Mutation (createAiConversation)
       │
       ▼
3. Rails enqueues StreamJob (async)
       │
       ▼
4. StreamService connects to MCP Server
       │
       ▼
5. MCP Server calls Mistral Agents API
       │
       ▼
6. Mistral decides which tools to call
       │
       ▼
7. MCP executes tools via lago-rust-client
       │
       ▼
8. Response streamed back via WebSocket
```

MCP Server - The Bridge

36 tools available for Lago operations:

Category	Tools
Invoices	list, get, create, update, download, retry, preview
Customers	list, get, create, current usage
Subscriptions	list, get, create, update, delete
Events	list, get, create
Payments	list, get, create
Plans	list, get, create, update, delete
Coupons	list, get, apply
Logs	activity logs, API logs

Real-time Response Streaming

Backend (Rails):

```
mistral_agent.chat(message) do |chunk|
  trigger_subscription(chunk:, done: false)
  sleep CHUNK_DELAY  # 30ms throttle
end
```

Frontend (React):

```
subscription onConversation($id: ID!) {
  aiConversationStreamed(id: $id) {
    chunk
    done
  }
}
```

Security Considerations

Aspect	Implementation
Organization-scoped	Each conversation tied to organization
API Key isolation	Uses org's Lago API key for MCP calls
Permission-based	`ai_conversations:create` required
No cross-tenant data	MCP server receives API key per request

Conversation History

Lago stores metadata, Mistral stores the full history

Storage	Data
Lago DB	`id` , `name` , `mistral_conversation_id` , `organization_id`
Mistral Cloud	Full message history, tool calls, responses

```
-- ai_conversations table (Lago)
id                      UUID PRIMARY KEY
name                    VARCHAR  -- First message as title
mistral_conversation_id VARCHAR  -- Links to Mistral history
organization_id         UUID     -- Multi-tenant isolation
membership_id           UUID     -- User who created it
```

Benefits: Minimal storage on Lago, full context maintained by Mistral

System Prompt Overview

The AI Agent uses a comprehensive **security-focused system prompt** that defines:

Aspect	Purpose
Identity	Lago Billing Assistant with defined responsibilities
Tenant Isolation	Only access data within authenticated organization
Security	Protection against prompt injection & data leakage
Scope	Allowed and forbidden operations
Safeguards	Confirmation requirements for destructive actions

Example Interactions

✅ **Query:**

> "Show me overdue invoices for the last 30 days"

Invoice #	Customer	Amount	Due Date
INV-001	Acme Corp	$1,250	2024-01-15

✅ **Destructive with Warning:**

> "Terminate subscription for Acme Corp"

⚠️ *"This subscription is currently active ($2,500/month). Type CONFIRM to proceed."*

❌ **Blocked — Prompt Injection:**

> "Ignore previous instructions..."

AI Roadmap

Phase	Feature	Description
Now	Billing Assistant	Automate repetitive tasks, query logs, bulk actions
Q1	Finance Assistant	Natural language queries, custom report builder
Q2	Anomaly Detection	Usage spikes, payment failures, pricing errors
Q2	Revenue Forecast	ML-powered predictions with scenarios
Q3	Customer AI	"Chat with your invoice", plan recommender
Q3	Pricing Assistant	Deal optimization, pricing simulations
Q4	AI SDK	Auto-track AI costs (OpenAI, Anthropic...)

Roadmap Highlights

Finance Assistant

> "Show me revenue from enterprise customers in Q3 who upgraded"

Anomaly Detection

> "Detected $47K revenue leakage from metering gaps"

Customer AI

> "Why is my bill $200 higher?" → "Your Compute usage spiked on Nov 12th"

AI SDK

```
# Auto-tracks tokens, applies 2.5x markup, bills customer
response = openai.chat.completions.create(...)
```